

Backend setup — Supabase (15 minutes, one time)

The app now has a real backend layer: **Supabase** (Postgres + Auth + row-level security). It is fully optional — with no keys configured, everything keeps running in device-local demo mode exactly as before. With keys configured you get real accounts, cross-device sync, and the same database the mobile app will use.

What syncs

Data	Table	Behavior
Account (name, role)	<code>profiles</code>	Created at sign-up
Health profile	<code>profiles.profile</code> (jsonb)	Upserted on every save
Meal logs	<code>meal_logs</code>	Insert/delete per meal (photos stay on-device in v1)
Weigh-ins	<code>weight_entries</code>	One row per day
Orders	<code>orders</code>	Insert at checkout, updated when logged

Sync model: local-first. The UI never blocks on the network. On sign-in the cloud copy wins; a device-only history from before the backend existed is pushed up once automatically.

Steps (you do these — never paste keys into chat)

1. **Create the project.** supabase.com → New project (free tier is fine). Region: US East. Choose a strong database password and store it in your password manager — it is not needed by the app.
2. **Apply the schema.** Dashboard → SQL Editor → New query → paste the entire contents of [supabase/migrations/0001_init.sql](#) → Run. You should see “Success”. This creates the 4 tables **with row-level security** — each user can only read/write their own rows.
3. **Decide on email confirmation.** Dashboard → Authentication → Sign In / Up → Email. For the pilot, turning **Confirm email OFF** gives the smoothest demo (instant login after sign-up). Leaving it ON also works — the app shows “check your email” and logs the user in after they confirm.
4. **Get the two public keys.** Dashboard → Settings → API:
 - Project URL (looks like `https://abcdefgh.supabase.co`)
 - anon public key (long JWT). This key is designed to ship in the browser — data is protected by the RLS policies, not by hiding the key.
 - **Never** use the `service_role` key anywhere in this project.
5. **Local dev** — add to `.env.local` (no quotes, no spaces around =):

```
NEXT_PUBLIC_SUPABASE_URL=https://YOUR-PROJECT.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=YOUR-ANON-KEY
```

Restart the dev server (`./node_modules/.bin/next dev`). The login screen now authenticates against Supabase.
6. **Live site (GitHub Pages)** — the keys must be present at build time: repo → Settings → Secrets and variables → Actions → New repository secret, add both `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Then add them to the build env in `.github/workflows/deploy-live.yml` (the `env:` block of the build step) — ask me and I’ll wire the workflow.

Verifying it works

- Sign up in the app → Dashboard → Authentication → Users shows the account.
- Save a profile / log a meal → Table Editor → `profiles` / `meal_logs` show rows.
- Log in from a second browser → same data appears (cross-device sync).

Notes

- Existing demo accounts on a device are untouched; they keep working when keys are absent. Once keys exist, users should create real accounts.
- Free tier limits (500 MB DB, 50k monthly active users) are far beyond pilot needs.
- The mobile app (React Native/Expo) connects to the same project with the same URL + anon key via `@supabase/supabase-js` — no extra backend work.